

```

# =====
# Student Name : Srushti Kale
# PRN          : 23070521067
# Experiment 4 : Regularization Techniques (L1/L2 and Dropout)
# =====

import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import regularizers
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# --- 1. Load and Preprocess Dataset ---
# Srushti Kale - 23070521067: Loading as pure numpy arrays to prevent DataFrame
housing = fetch_california_housing()
X = housing.data
y = housing.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Hyperparameters for training
EPOCHS = 100
BATCH_SIZE = 64

# --- 2. Baseline Model (Prone to Overfitting) ---
# Srushti Kale - 23070521067: Creating a large network to intentionally overfit
baseline_model = keras.Sequential([
    keras.layers.Dense(256, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dense(1)
])

baseline_model.compile(optimizer='adam', loss='mse')

print("Training Baseline Model...")
baseline_history = baseline_model.fit(
    X_train_scaled, y_train, epochs=EPOCHS, batch_size=BATCH_SIZE,
    validation_split=0.2, verbose=0
)

# --- 3. Model with L2 Regularization ---
# Srushti Kale - 23070521067: Applying L2 Ridge penalty to weights
l2_model = keras.Sequential([
    keras.layers.Dense(256, activation='relu', kernel_regularizer=regularizers.L2(0.01)),
    keras.layers.Dense(256, activation='relu', kernel_regularizer=regularizers.L2(0.01)),
    keras.layers.Dense(256, activation='relu', kernel_regularizer=regularizers.L2(0.01)),
    keras.layers.Dense(1, kernel_regularizer=regularizers.L2(0.01))
])

```

```

        keras.layers.Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
        keras.layers.Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.01)),
        keras.layers.Dense(1)
    ])

l2_model.compile(optimizer='adam', loss='mse')

print("Training L2 Regularization Model...")
l2_history = l2_model.fit(
    X_train_scaled, y_train, epochs=EPOCHS, batch_size=BATCH_SIZE,
    validation_split=0.2, verbose=0
)

# --- 4. Model with Dropout ---
# Srushti Kale - 23070521067: Applying Dropout layers to randomly deactivate neurons
dropout_model = keras.Sequential([
    keras.layers.Dense(256, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(1)
])

dropout_model.compile(optimizer='adam', loss='mse')

print("Training Dropout Model...")
dropout_history = dropout_model.fit(
    X_train_scaled, y_train, epochs=EPOCHS, batch_size=BATCH_SIZE,
    validation_split=0.2, verbose=0
)

# --- 5. Compare Performance ---
# Srushti Kale - 23070521067: Visualizing Validation Loss (MSE) to compare overfitting reduction
plt.figure(figsize=(10, 6))

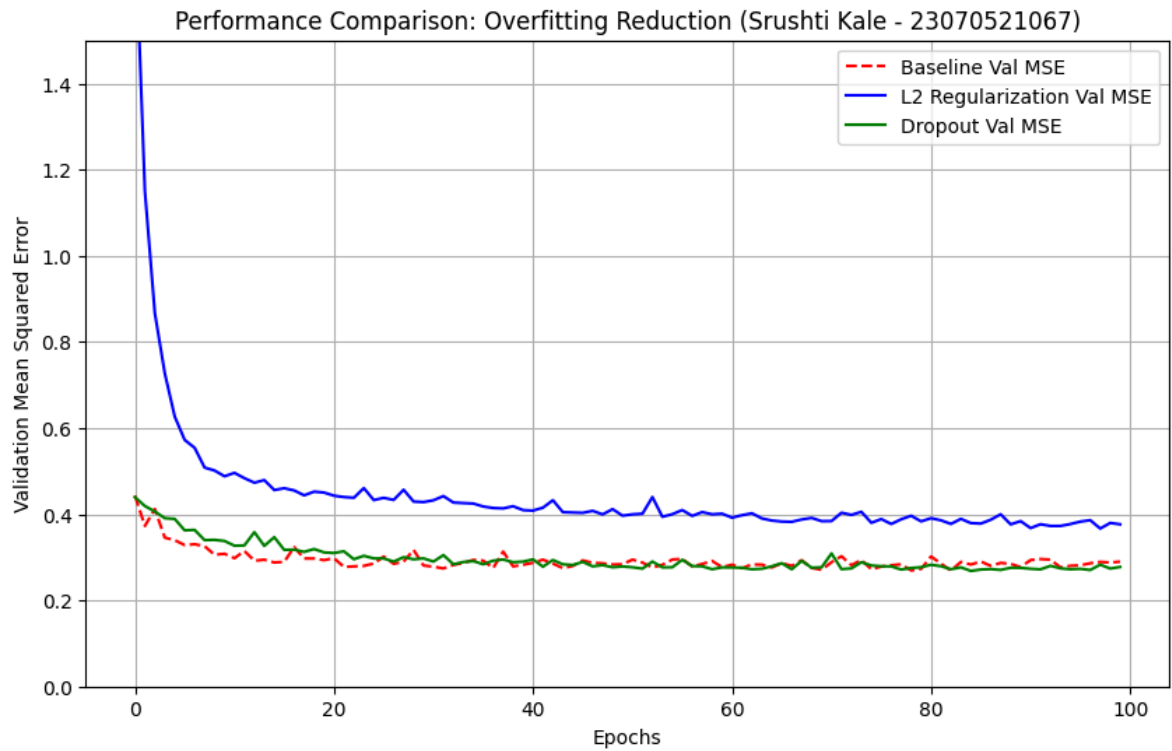
# Using 'val_loss' is safer across all Keras versions since our loss is 'mse'
plt.plot(baseline_history.history['val_loss'], label='Baseline Val MSE', color='blue')
plt.plot(l2_history.history['val_loss'], label='L2 Regularization Val MSE', color='orange')
plt.plot(dropout_history.history['val_loss'], label='Dropout Val MSE', color='green')

plt.title('Performance Comparison: Overfitting Reduction (Srushti Kale - 23070521067)')
plt.xlabel('Epochs')
plt.ylabel('Validation Mean Squared Error')
plt.ylim([0, 1.5])
plt.legend()
plt.grid(True)
plt.show()

# Evaluate on final test set
print("\n--- Final Test Set Evaluation (MSE) ---")
print(f"Baseline Test MSE : {baseline_model.evaluate(X_test_scaled, y_test, verbose=0):.4f}")
print(f"L2 Model Test MSE : {l2_model.evaluate(X_test_scaled, y_test, verbose=0):.4f}")
print(f"Dropout Test MSE : {dropout_model.evaluate(X_test_scaled, y_test, verbose=0):.4f}")

```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: U
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Training Baseline Model...
Training L2 Regularization Model...
Training Dropout Model...
```



```
--- Final Test Set Evaluation (MSE) ---
Baseline Test MSE : 0.2868
L2 Model Test MSE : 0.3651
Dropout Test MSE  : 0.2599
```

